Dynammic Programming in Rook Endgames

Melissa Osheroff, Tien Nguyen, Raaghav Thirumaligai

March 5, 2026

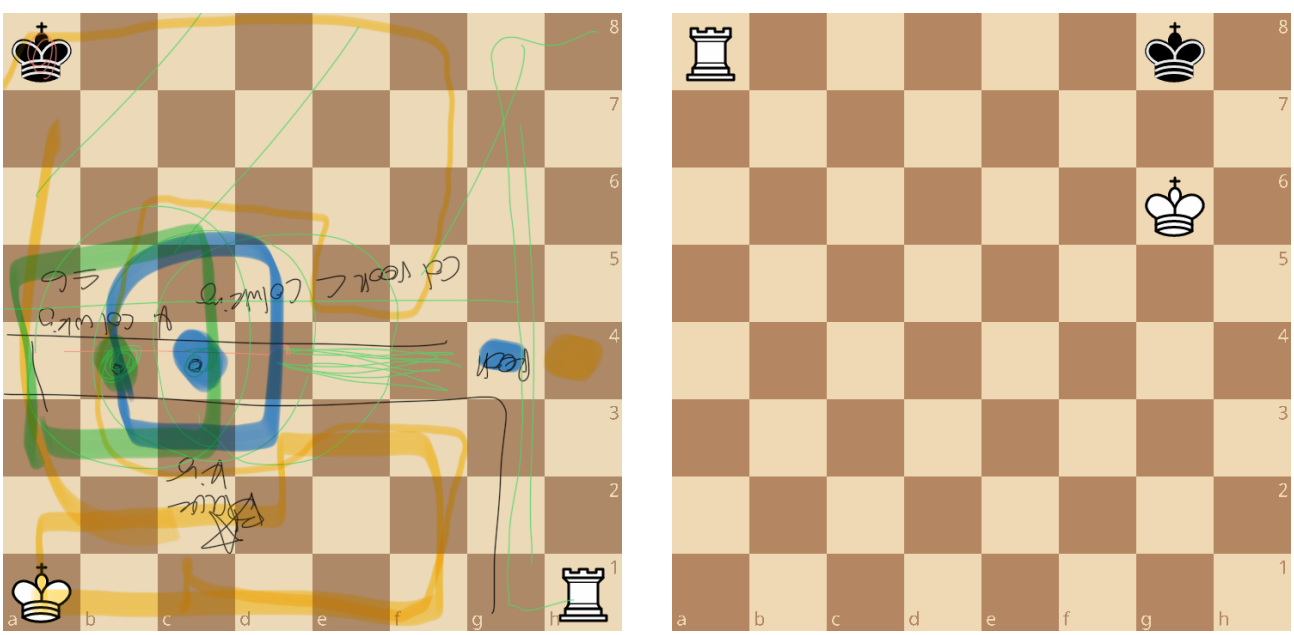
Abstract

This project proposes the software implentation of a dynamic program that finds the best outcome for Player 1 in chess endgame. Player 1 has a rook and a king left, whereas Player 2 has only a king. Thus, this is a zero-sum game where Player 1 is trying to checkmate, and Player 2 is trying to stalemate. We will start by hard-coding the starting positions for each piece. If easily solvable, we will try to automate the program using defined chess rules to find the most optimal path for random starting positions. The results of this project will be presented and discussed in a comprehensive report.

1 Problem Statement

System Description

We want to find the most optimal path to winning a game of chess, given a rook(H1) and a king(A1) against a king(A8). The optimal path is defined as a checkmate occuring before stalemate for Player 1 and occurring with the least amount of moves possible.



(a) Initial Position (b) Checkmate Position

Figure 1: Rook and King versus King Endgame from possible initial position to an example mate.

2 Proposed Approach

We propose the following steps:

1. **Define Action Spaces:** Define the sets of all possible positions for all pieces on the board. This will be unique to each turn.

2. **Define Legal Moves:** Restrict action spaces to allow only legal positions (according to chess rules).

3. **Building the Graph:** We construct a directed graph where each node represents a valid board configuration together with the current player to move. Edges correspond to legal moves under standard chess rules. Terminal nodes are classified as checkmate, stalemate, or 50-move termination. The terminal payoff is defined as

$$J = o + \epsilon\, m,$$

   where $o \in \{1, 0, -1\}$ represents stalemate, draw by 50-move rule, or checkmate respectively, m is the number of moves taken to reach the terminal state, and $\epsilon > 0$ is a small weighting parameter to prefer faster mates.

4. **Backtracking:** After the graph is computed, find global minima. Then traversing back up the tree will give the optimal set of moves to reach one such minima.

3 Cases

We distinguish the following terminal cases:

Checkmate:	Player 1 successfully forces mate. The value of the state is -1 (adjusted by move penalty).
50-Move Rule:	If 50 moves occur without capture or checkmate, the game is declared a draw with value 0.
Stalemate:	Player 2 has no legal moves but is not in check. The value of the state is $+1$.
Draw:	Player 2 captures the rook, the game is over, and neither king can progress. The value of the state is $+1$.

These cases define the boundary conditions for the dynamic programming recursion.